1. Scenario: Word Cloud for Popular Keywords

Question: You're given a collection of product reviews. How will you create a word cloud to visualize the most frequent keywords?

Answer:

```
from wordcloud import WordCloud

import pandas as pd

import matplotlib.pyplot as plt

text = """

The movie was fantastic! Acting was brilliant and the story was well-written.

Amazing visuals and a gripping plot.

It was a bit slow in the middle, but overall a great movie experience.

"""

wc = WordCloud(background_color='white').generate(text)

# Display the word cloud

plt.figure(figsize=(8, 4))

plt.imshow(wc, interpolation='bilinear')

plt.axis('off')

plt.title("Word Cloud Output", fontsize=16)

plt.show()
```

2. Scenario: Sentiment Analysis for Hotel Reviews

Question: A hotel wants to understand how customers feel about their service. How would you apply sentiment analysis to the reviews?

Answer:

```
from nltk.sentiment.vader import SentimentIntensityAnalyzer

import pandas as pd
```

HOPE AI

```
data = pd.DataFrame({'review': [
        'The hotel was excellent and the staff were very friendly.',
        'The room was dirty and the bathroom smelled terrible.',
        'The food was delicious and the service was outstanding.',
        'I had a terrible experience. The bed was uncomfortable.',
        'Very disappointing. The air conditioner was not working.',
        'The location was convenient and the hotel was peaceful.',
        'Poor service, overpriced food, and noisy surroundings.',
        'The room was spacious and the bed was comfortable.',
        'Check-in was slow, but the staff were helpful.',
        'The hotel was not worth the money.'
    ] })
sid=SentimentIntensityAnalyzer()
data['Scores']=data['review'].apply(lambda commenttext: sid.polarity_scores(commenttext))
data['Compound']=data['Scores'].apply(lambda score_dict: score_dict['compound'])
data['Negative']=data['Scores'].apply(lambda score_dict: score_dict['neg'])
data['Positive']=data['Scores'].apply(lambda score_dict: score_dict['pos'])
data['Neutral']=data['Scores'].apply(lambda score_dict: score_dict['neu'])
data['comp_score']= data['Compound'].apply(lambda c: 'pos' if c >=0 else 'neg')
posneg=pd.DataFrame(data['comp_score'].value_counts())
print(data)
print(posneg)
```

3. Scenario: Topic Modeling for News Articles

Question: You're given thousands of news articles with no labels. How would you identify the main themes or topics?

HOPE AI

Answer:

```
df = pd.DataFrame({
    'article': [
        """Artificial intelligence is transforming healthcare.
        Machine learning algorithms help doctors detect diseases.
        Deep learning models analyze medical images and scans.
        AI can support early diagnosis and personalized treatment.""",
        """Machine learning helps businesses analyze large datasets.
        Supervised learning uses labeled data for prediction.
        Unsupervised learning discovers hidden patterns.
        Algorithms such as decision trees and random forests
        are widely used for predictive analytics."""]})
from sklearn.feature_extraction.text import CountVectorizer
vectorizer = CountVectorizer(stop_words='english')
dtm = vectorizer.fit_transform(df['article'])
from sklearn.decomposition import LatentDirichletAllocation
# Initialize and fit Topic Model
lda = LatentDirichletAllocation(n_components=1, random_state=42)
lda.fit(dtm)
feature_names = vectorizer.get_feature_names_out()
for index,topic in enumerate(lda.components_):
    results=([feature_names[i] for i in topic.argsort()[-10:]])
    print(results)
```

4. Scenario: Feature Extraction Using CountVectorizer

Question: You want to feed customer messages into a machine learning model. How do you convert the text into numerical features using CountVectorizer?

HOPE AI

```
df = pd.DataFrame({
    'article': [
        """Machine learning helps businesses analyze large datasets.
        Supervised learning uses labeled data for prediction.
        Unsupervised learning discovers hidden patterns.
        Algorithms such as decision trees and random forests
        are widely used for predictive analytics."""]})
from sklearn.feature_extraction.text import CountVectorizer
vectorizer = CountVectorizer(stop_words='english')
dtm = vectorizer.fit_transform(df['article'])
print(dtm)
```

5. Scenario: Choosing Between CountVectorizer and TF-IDF

Question: When analyzing job descriptions, should you use CountVectorizer or TF-IDF Vectorizer? Why?

Answer:

CountVectorizer - Counts how many times each word appears in each document.

TF-IDF Vectorizer-Gives higher importance to words that are frequent in a particular document but less common across the entire collection of documents.

I start with TF-IDF for job-description analysis because it reduces the influence of terms that appear frequently across the descriptions and highlights more distinctive skills and technologies. However, if my objective is to measure the actual frequency of requested skills, I would use CountVectorizer or word-frequency analysis.

6. Scenario: Using TF-IDF to Find Important Terms in a Resume

Question: You want to extract the most unique and relevant words from a resume. How would TF-IDF help?

Answer:

HOPE AI

“TF-IDF identifies terms that are frequent in a particular resume but less common across the document collection. It helps highlight distinctive skills and technologies while reducing the weight of common terms. However, uniqueness alone doesn't guarantee job relevance, so I would calculate TF-IDF on the combined collection of resumes and job descriptions, then compare their vectors using cosine similarity.

HOPE AI